HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Introduction to Constraint Programming

Slide based on slides from the course CP given by Pascal Van Hentenryck & Yves Deville

ONE LOVE. ONE FUTURE.

Content

- A first challenge
- Constraint Satisfaction Problem
- Constraint Programming
- Examples with Or-Tools

Sudoku

- Fill the empty cells with numbers in {1,..,9}
- **Constraint** : each number only appears once in each
  - Row
  - Column
  - Region

easy

		1				8	9	
	2	7			9		5	
		4		8	2			
	6		9	2		1	4	
				5				
	9	8		6	1		3	
			2	1		4		
	1		7			3	6	
	7	9				2		

hard

5	9				7		8	
			5					7
4	1		8			5		6
		1	3					4
		5				9		
8					4	2		
9		2			3		4	5
1					5			
	5		4				6	9

Sudoku

easy

		1				8	9	
	2	7			9		5	
		4		8	2			
	6		9	2		1	4	
				5				
	9	8		6	1		3	
			2	1		4		
	1		7			3	6	
	7	9				2		

6	3	1	5	7	4	8	9	2
8	2	7	1	3	9	6	5	4
9	5	4	6	8	2	7	1	3
7	6	5	9	2	3	1	4	8
1	4	3	8	5	7	9	2	6
2	9	8	4	6	1	5	3	7
3	8	6	2	1	5	4	7	9
4	1	2	7	9	8	3	6	5
5	7	9	3	4	6	2	8	1

hard

5	9				7		8	
			5					7
4	1		8			5		6
		1	3					4
		5				9		
8					4	2		
9		2			3		4	5
1					5			
	5		4				6	9

5	9	6	1	4	7	3	8	2
3	2	8	5	9	6	4	1	7
4	1	7	8	3	2	5	9	6
2	7	1	3	8	9	6	5	4
6	4	5	2	7	1	9	3	8
8	3	9	6	5	4	2	7	1
9	6	2	7	1	3	8	4	5
1	8	4	9	6	5	7	2	3
7	5	3	4	2	8	1	6	9

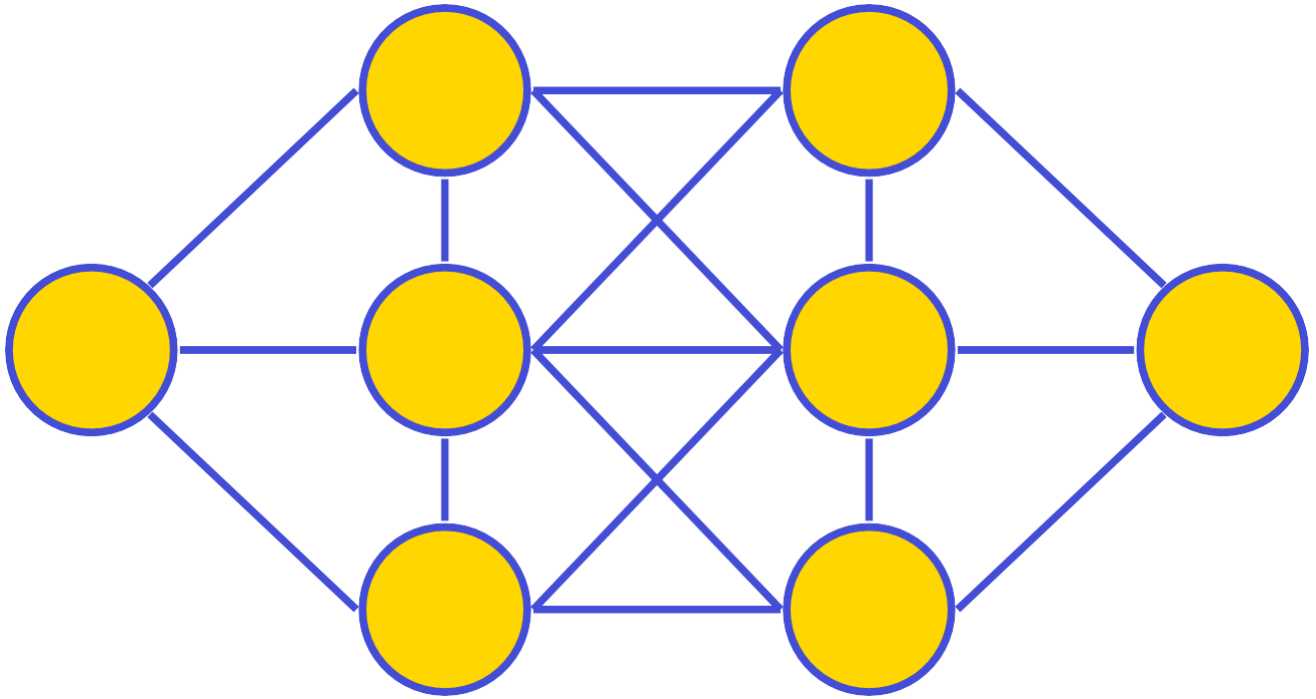
Sudoku

- Sudoku is a (very) simple example of Constraint Satisfaction Problem (CSP)
  - A set of variables, defined over domains
  - A set of constraints over the variables
- What is a solution?
  - A solution is an assignment of values to the variables
  - does not violate any constraint

A challenge from T. Walsh

- Place numbers 1 through 8 on nodes
  - Each number appears exactly once
  - No connected nodes have consecutive number

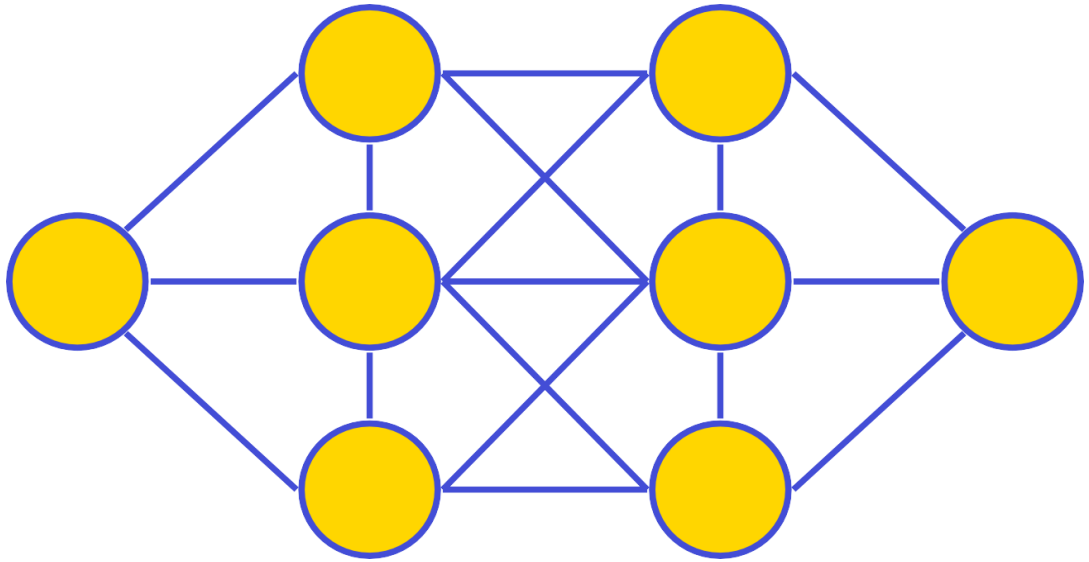
How long
will you take ?



- This is an example of a Constraint Satisfaction Problem (CSP)
  - A set of variables defined over domains
  - A set of constraints over the variables
- It can be solved by Constraint Programming

How did you solved this problem manually ?

Which techniques did you used ?



Content

- A first challenge
- **Constraint Satisfaction Problem**
- Constraint Programming
- Examples with Or-Tools

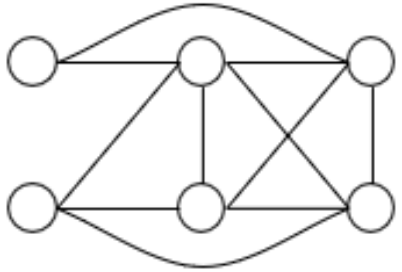
Constraint Optimization Problem

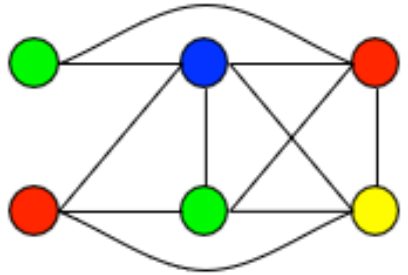
- Constraint Satisfaction Problem (CSP)
  - A set of variables, defined over finite domains
  - A set of constraints over the variables
- Solution
  - A solution is an assignment of values to the variables
  - No violation of constraint
- Constraint Optimization Problem (COP)
  - Add an objective function
  - Find a solution maximizing the objective function

Many CSP / COP are complex and challenging (NP-hard)

A simple example

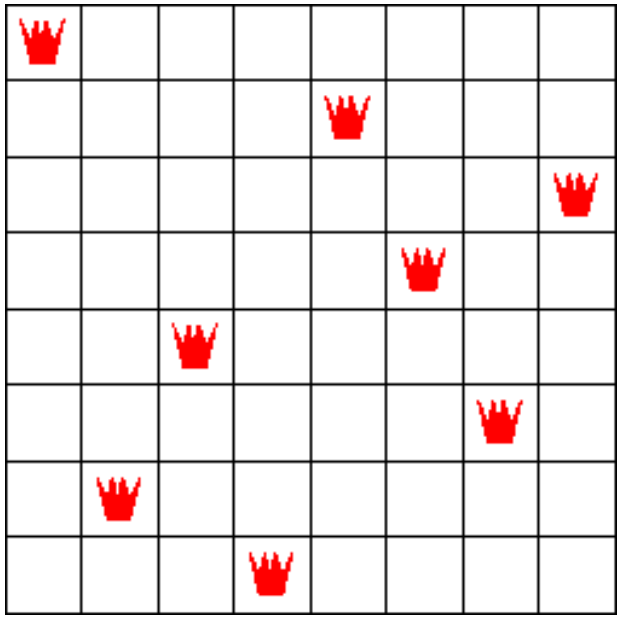
- **CSP** : Graph Coloring with 4 colors
  - Graph G=(V,E)
  - Decision variables
    - color[v] v < V
  - Domain of variables : {red, blue, green, yellow}
  - Constraints
    - 6(v,w) < E : **color[v] ≠ color[w]**
- **COP** : minimize the used colors

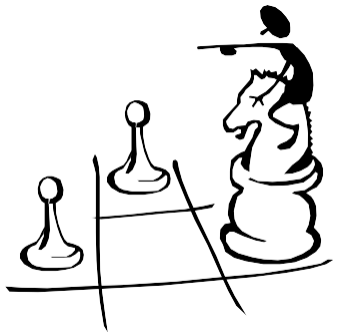




Example : N-queens

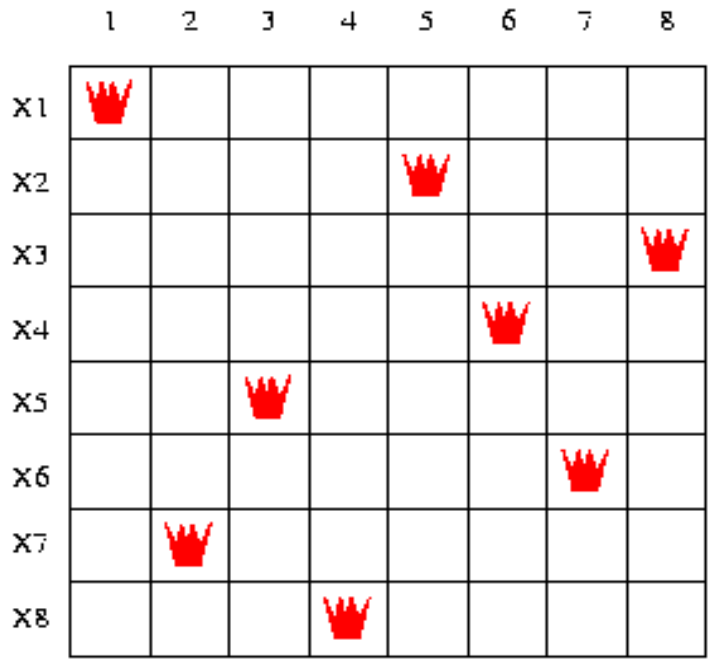
- How to place 8 queens on a 8x8 board such that they do not attack each other
- A solution





N-queens as a CSP

- **Variables**
  - X_i : position (column) of the queen on row i ($1 \leq i \leq 8$)
- **Domains**
  - Domain of X_i :
    - {1,2,3,4,5,6,7,8}



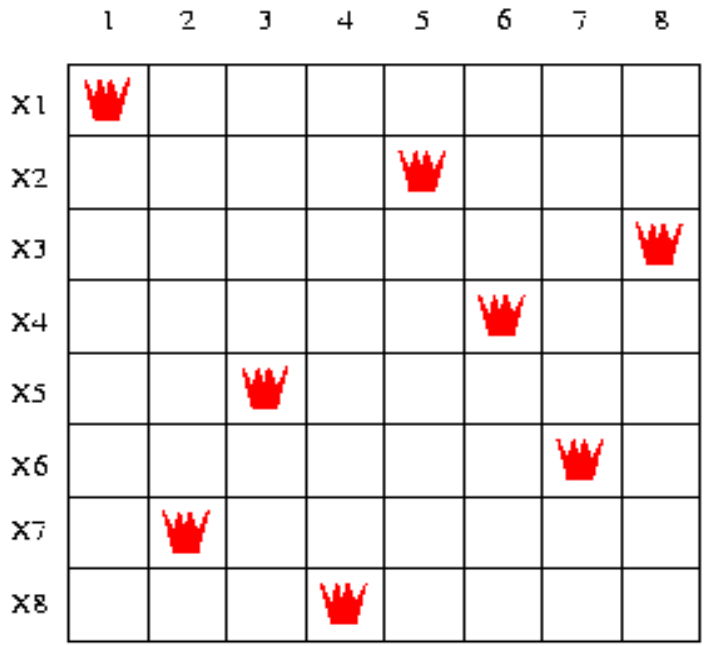
N-queens as a CSP

- **Constraints**
  - Two queens cannot be on the same column
    - $X_i \neq X_j$
  - Two queens cannot be on the same diagonal
    - $X_i - X_j \neq i - j$ $X_i - X_j \neq j - i$
- 8-queens
  - 8 variables
  - 84 contraints

X1≠X2, X1≠X3, X1≠X4, X1≠X5, X2≠X3, X2≠X4, X2≠X5, X3≠X4, X3≠X5, X4≠X5, …
X1-X2≠-1, X1-X2≠1, X1-X3≠-2, X1-X3≠2, X1-X4≠-3, X1-X4≠3, X1-X5≠-4, X1-X5≠4, X2-X3≠-1, X2-X3≠1, X2-X4≠-2, X2-X4≠2, X2-X5≠-3, X2-X5≠3, X3-X4≠-1, X3-X4≠1, X2-X5≠-2, X3-X5≠2, X4-X5≠-1, X4-X5≠1, …

N-queens as a CSP

- X_i : position of the queen on row i
  - $X_1 = 1$
  - $X_2 = 5$
  - $X_3 = 8$
  - $X_4 = 6$
  - $X_5 = 3$
  - $X_6 = 7$
  - $X_7 = 2$
  - $X_8 = 4$



Constraints

- Constraints as basic computational elements
  - $2X + 3Y \leq 17$ $\quad X^2 - Y^3 = 17$
- Constraints appear naturally in :
  - artificial intelligence
  - operational research
  - combinatorial optimization
  - graphics, visualisation
  - concurrency
  - databases
  - hardware design, verification
  - ...

Who does optimization ?

- Microsoft (1st largest software company)
- SAP (2nd largest software company)
- Oracle (3rd largest software company)
  - Supply chain management
- IBM (of course)
- Siebel, PeopleSoft
  - Personal management
- UPS
  - Dispatching
- Google, IBM, Carmen systems, Fair Isaac

Complexity of CSP

- Computational Complexity
  - NP-Hard or worse (at least exponential)
  - No reasonable approximation
- *Yet, it is important to solve them*
- The best specific algorithms
  - tough to design and implement
  - experimental

Solving Paradigms

- **Mathematical Programming** : *IP, MIP, column generation, ...*
- **Dynamic Programming**
- **Local Search** : tabu search, simulated annealing, genetic algorithms, evolutionary algorithms...
- Greedy algorithms, Ant Colony Optimization, greedy randomized, ...
- **Constraint Programming**
- ...

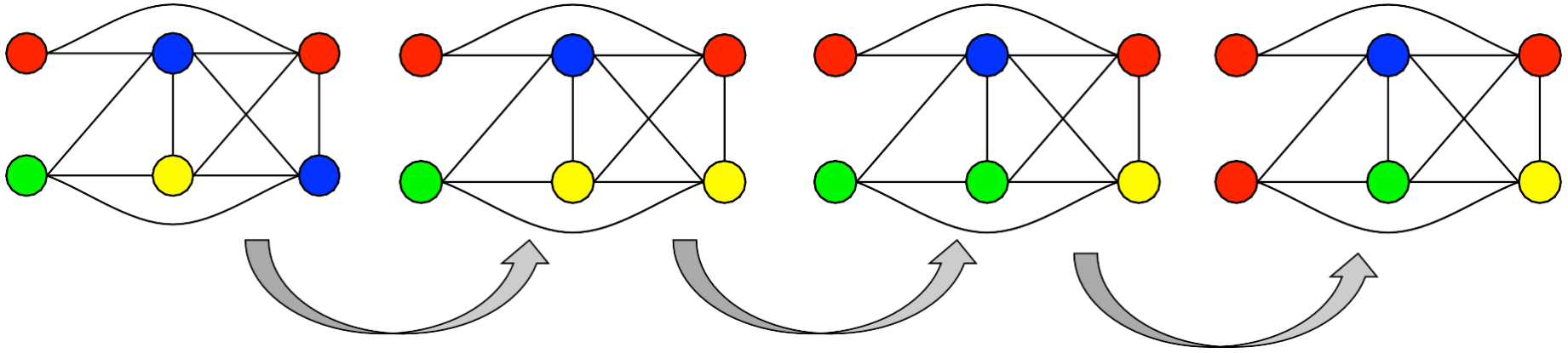
Solving Paradigms

Paradigms can usually be classified according to two criteria

- **Perturbative** vs **constructive**
- **Systematic** vs **incomplete**

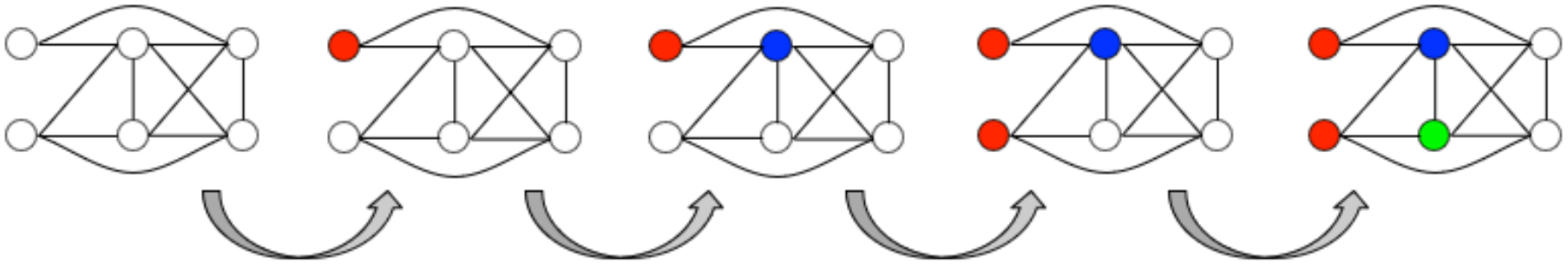
Perturbative vs Constructive

- **Perturbative** approaches
  - A candidate solution is a complete solution (a value is assigned to each variable)
  - Generation of a new solution by modifications, pertubation of an existing one
  - Search is performed on the succession of candidate solutions



Perturbative vs Constructive

- **Constructive** approaches
  - A candidate solutions is partial (some variables are not assigned)
  - A partial candidate solution is progressively extended, constructed



Systematic vs Incomplete

- **Systematic** approaches
  - Systematic exploration of the search space
  - Completeness
    - CSP : find an *exact* solution
    - COP : find the *best* solution, (+ proof of optimality)
    - If no solution, proof of non-existence
  - Computationally expensive

Systematic vs Incomplete

- **Incomplete** approaches
  - Exploration based on a neighborhood function
  - Incomplete
    - CSP: find an *approximation* of a solution
    - COP: find a *good* solution
    - No guarrantee to find the (best) solution
  - Computationally less expensive

Paradigms

	Systematic	Incomplete
Pertubative	▪ Simplex ▪ Systematic local search	▪ **Local search** ▪ Evolutionary algo.
Constructive	▪ Dynamic Programming ▪ Branch & Bound ▪ **Constraint Programming**	▪ Greedy ▪ Ant Colony Optimizatio ▪ GRASP

Content

- A first challenge
- Constraint Satisfaction Problem
- **Constraint Programming**
  - **What is CP ?**
  - Constraints and propagation
  - Modeling
  - Searching
  - Global constraint
- Examples with Or-Tools

Constraint programming

- A field at the intersection of
  - Artificial intelligence
  - Operational research
- A framework for solving CSP

Constraint-Based Modelling

Solution process
=
Model + Search

- The model
  - what the solutions are and their quality
- The search
  - how to find the solutions

CP and CBLS

- **Constraint Programming**
  - *Modeling with Constraints*
  - Systematic and constructive paradigm
  - Computation based on **branch & propagate**
- **Constraint-Based Local Search**
  - *Modeling with Constraints*
  - Incomplete and perturbative paradigm
  - Computation based on **neigborhood**

The CP computation model

- Iterations of two steps
- **Propagation**
  - reducing the domains of the variables
  - uses constraints to detect impossible values of variables
  - solve a relaxation of the problem
- **Branch** (also called **Search**)
  - Decompose into subproblems
  - nondeterministic choices (e.g., giving a value for a variable)
  - automatic support for backtracking

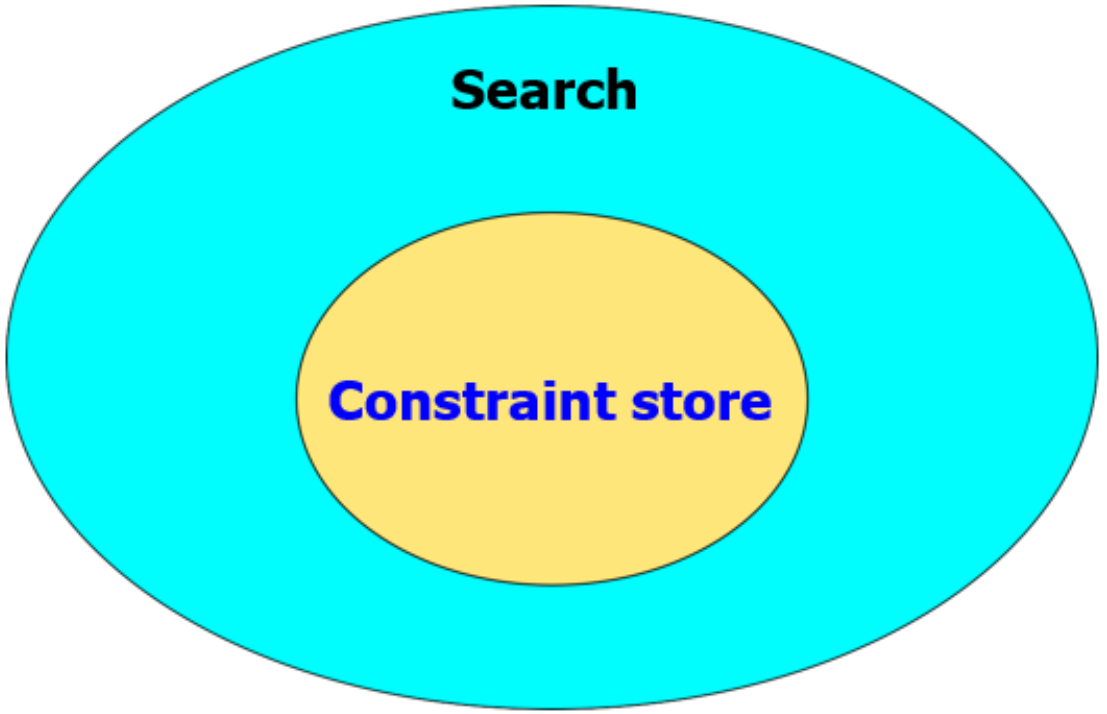
Constraint Programming

- Began in 1980s from AI world
  - Prolog III (Marseilles, France)
  - CLP(R) (Melbourne & IBM)
  - CHIP (ECRC, Germany)
- Active research area
  - Specialized conferences (CP, CP-AI-OR, …)
  - Journal (Constraints, CP Letters)
  - Commercial products

The CP Language

- A rich constraint language
  - Arithmetic, higher-order, logical constraints
  - Global constraints for natural substructures
- Specification of a search procedure
  - Definition of search tree to explore
  - Specification of exploration strategy
- Separation of concerns
  - Constraints and search are separated

Computational Model



Domains of computation

Constraint propagation techniques are related to specific **computation domains**

- Finite domains
- Finite sets
- Boolean
- Interval
- Reals
- Graphs
- …

Content

- A first challenge
- Constraint Satisfaction Problem
- **Constraint Programming**
  - What is CP ?
  - **Constraints and propagation**
  - Modeling
  - Searching
  - Global constraint
- Examples with Or-Tools

What is a constraint ?

- Numerical inequalities and equations
- Combinatorial / global constraints
  - natural subproblems arising of many applications
  - e.g. a set of activities A cannot overlap in time
- Logical/threshold combinations of these
- …

Constraints

- Powerful modelling tool
  - Numerical constraints
    - $x = y + 2z$, $x \neq y - 2$, $x \leq y - z$, $|x-y| \neq 2$, ...
  - Symbolic Constraints
    - $cost = \sum_{i<R} price[rack[i]]$
  - Composed constraints
    - $\sum_{1 \leq i \leq k} (x[i]=y) \leq k$ (at most k occurences of y in x[])
    - $load[j] = \sum_{i<Obj} (bin[i]=j).w[i]$ (load of bin j in bin packing)
  - Global constraints
    - alldifferent(x[1..n])
    - knapsack(x[1..n], weigth[1..n], cap)
    - Grammar constraints, scheduling constraints, ...

Propagation

Why propagation ?

- Exhaustive search of a solution: exponential time

Objectives of propagation

- Reduce the search space
- Find an equivalent CSP to the original one with *smaller domains* of variables

Techniques

- Consider the constraints *locally*
- **Consistency techniques**
  - Domain Consistency : consider each possible value in the domain
  - Bound Consistency : only considers the bounds of the domains
  - Other consistencies

Pruning

- Domain Consistency
  - The holy grail
- A constraint is **domain-consistent** if,
  - for every variable x and every value in the domain of x,
  - there exist values in the domains of the other variables
  - such that the constraint is satisfied for these values

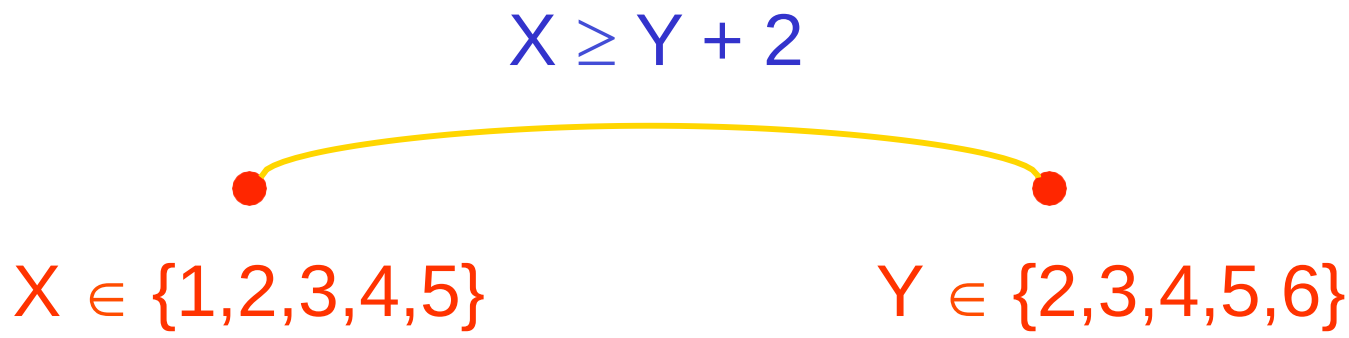
Domain consistency

$$X \geq Y + 2$$

$X \in \{1,2,3,4,5\}$ $Y \in \{2,3,4,5,6\}$

- remove any values from the domain of X for which there is no value in the domain of Y that satisfies the constraint (and vice versa)

Domain consistency



- remove any values from the domain of X for which there is no value in the domain of Y that satisfies the constraint (and vice versa)

Domain consistency

$$X \geq Y + 2$$

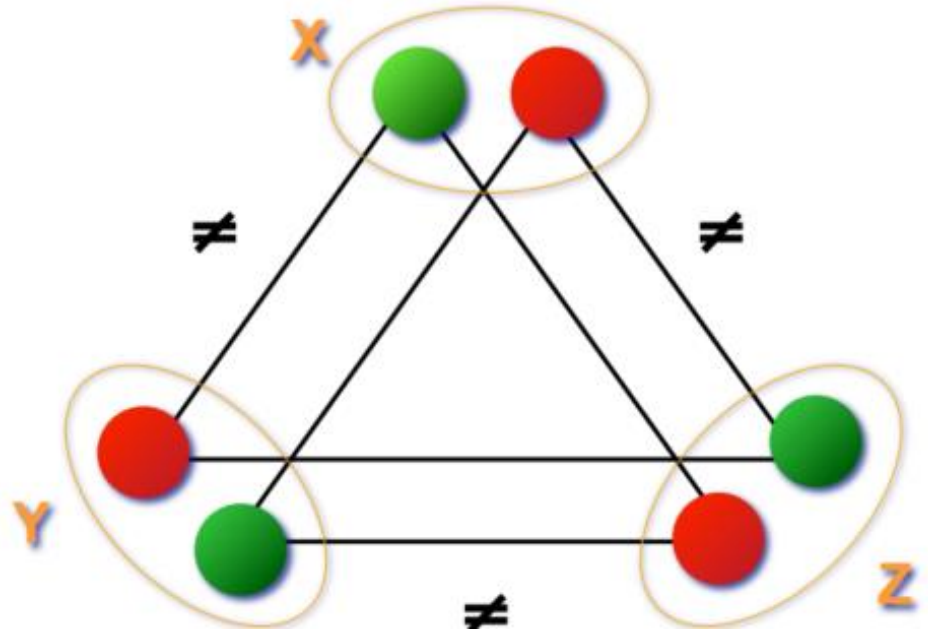
$X \in \{4,5\}$ $Y \in \{2,3\}$

- This constraint is now domain consistent
- Possible objective : make *all* constraints domain consistent
- Should be done *efficiently*

Consistency does not solve the CSP !

Domain consistency

- Example:
  - x, y, z = {1, 2}
  - x ≠ y, y ≠ z, x ≠ z



Domain consistency

- Example
  - x = 3y + 5z
  - D(x) = {2, 3, 4, 5, 6, 7}, D(y) = {0, 1, 2}, D(z) = {-1, 0, 1, 2}
- The Domain Consistency:
  - D(x) = {3, 5, 6}, D(y) = {0, 1, 2}, D(z) = {0, 1}

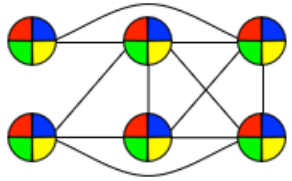
Bound Consistency

- Achieving domain consistency can be computationally expensive
- Bound consistency is a weaker form of consistency
  - a domain is approximated by an interval [min D(x), max D(x)]
  - only verify that the bounds of the domain are in a solution of the constraint

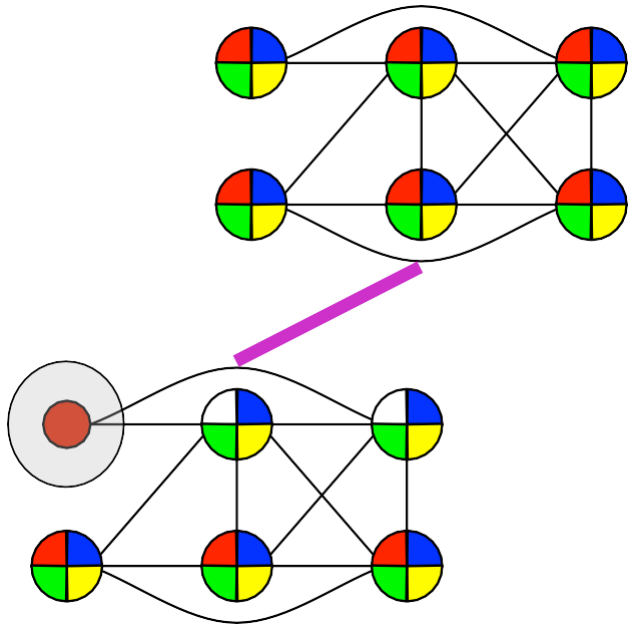
Bound Consistency

- Specialized to each constraint type
  - x + y + z = 10
  - x in {0,1,2,3,4}, y in {1,2,4}, z in {1,2,4}
- Domain consistency (AC) gives
  - x in {2,4}, y in {2,4}, z in {2,4}
  - expensive to compute ($O(d^3)$)
- Bound consistency (BC) gives
  - x in in {2,3,4}, y in {2,4}, z in {2,4}
  - can be computed very efficiently ($O(1)$)

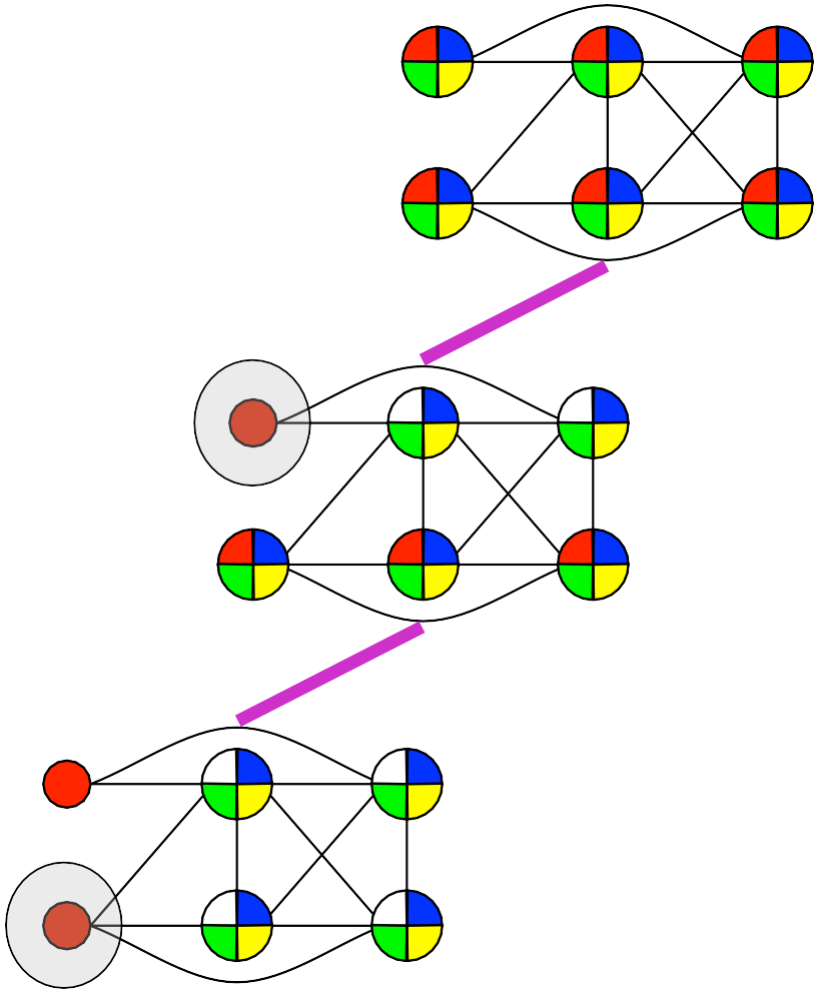
Graph Coloring CP



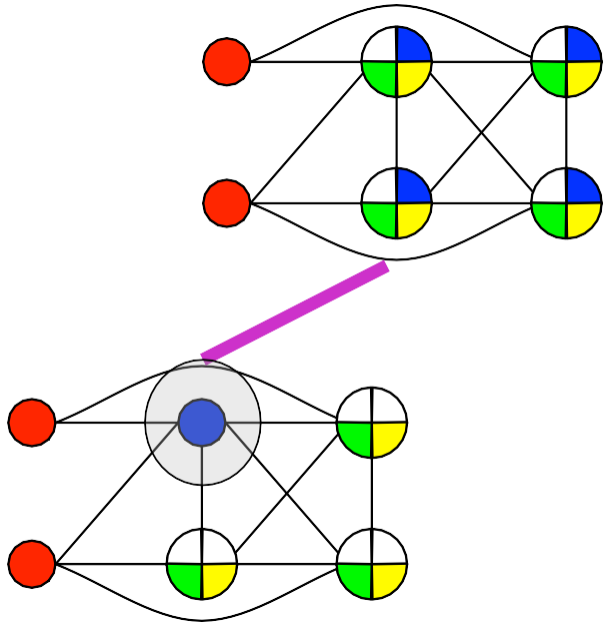
Graph Coloring CP



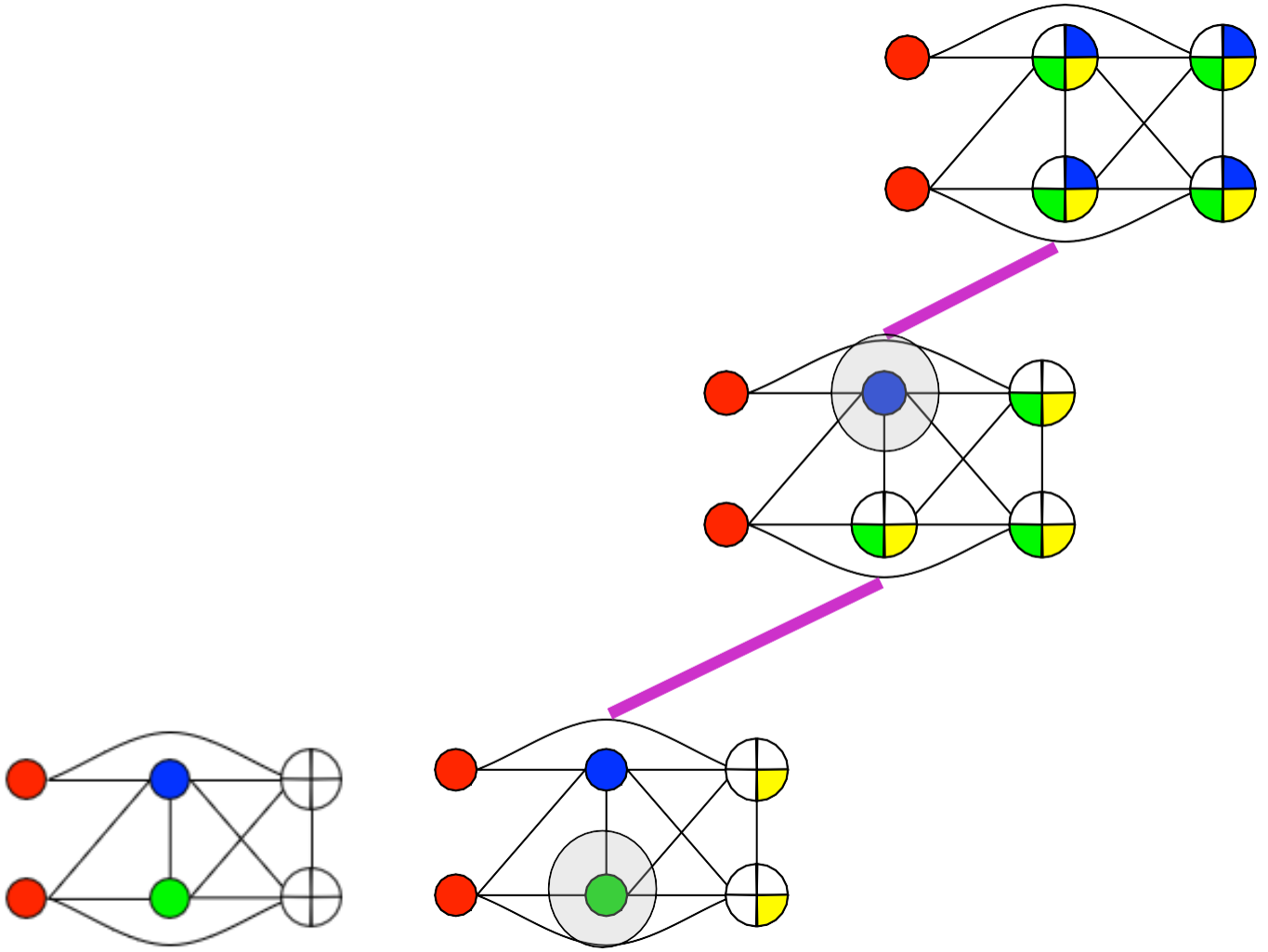
Graph Coloring CP



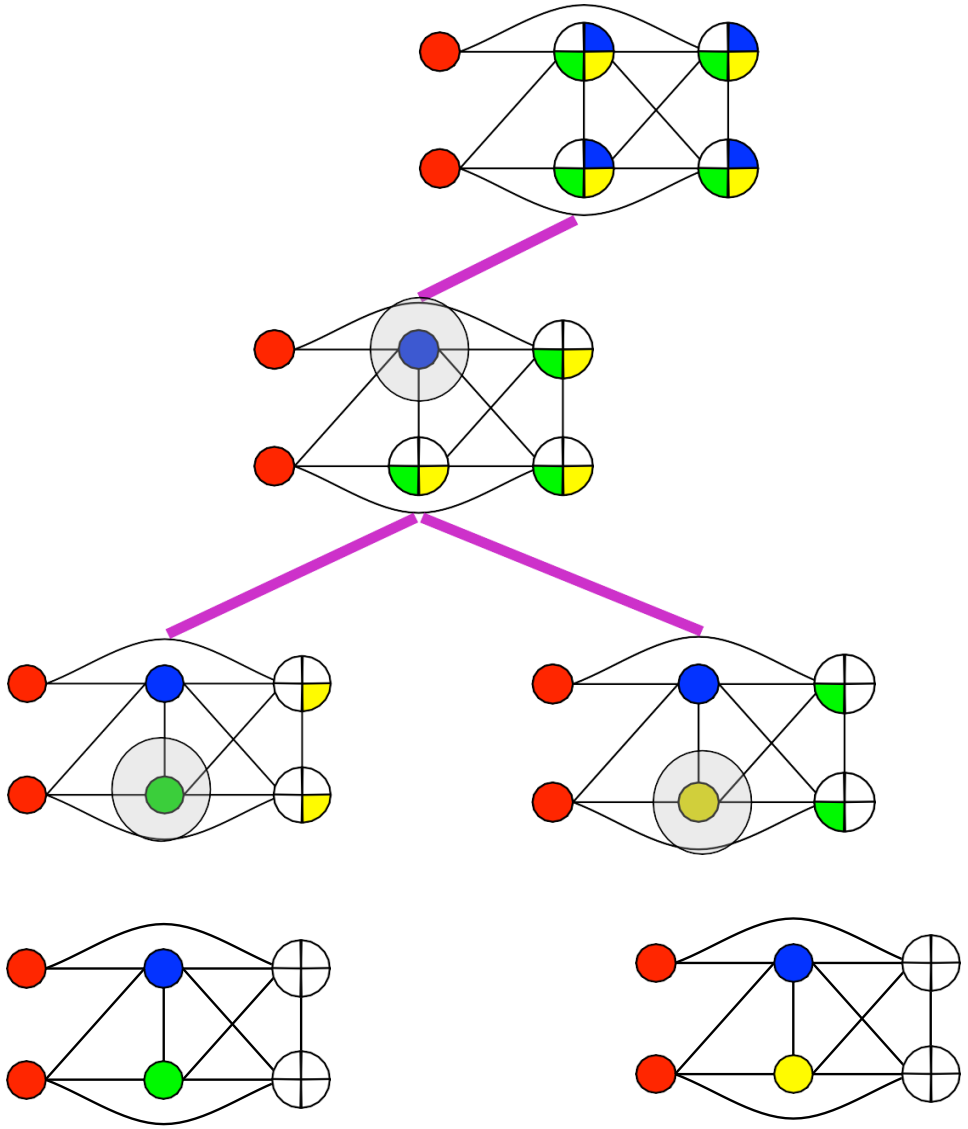
Graph Coloring CP



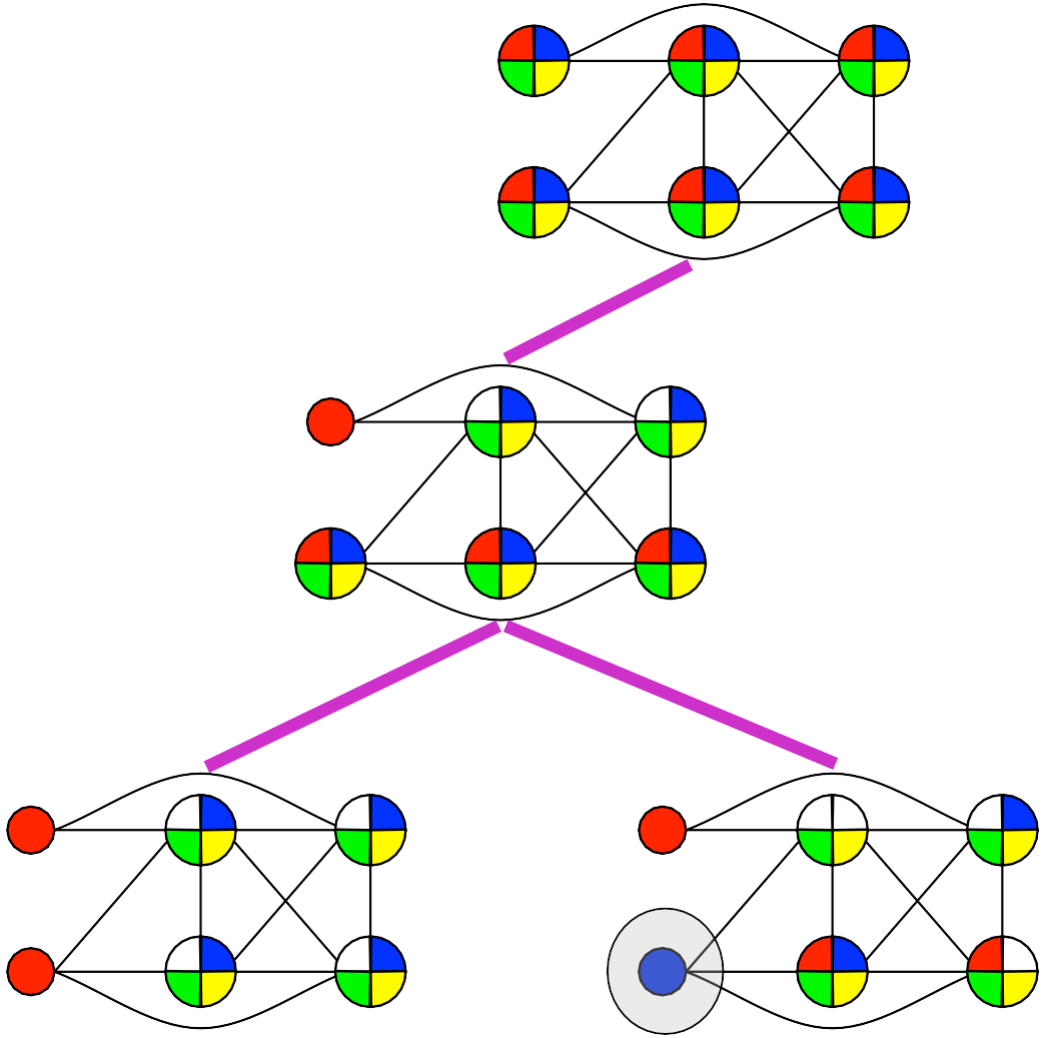
Graph Coloring CP



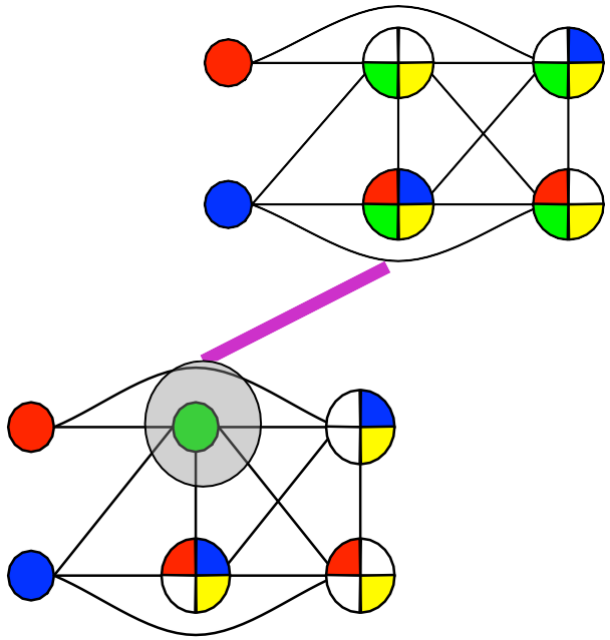
Graph Coloring CP



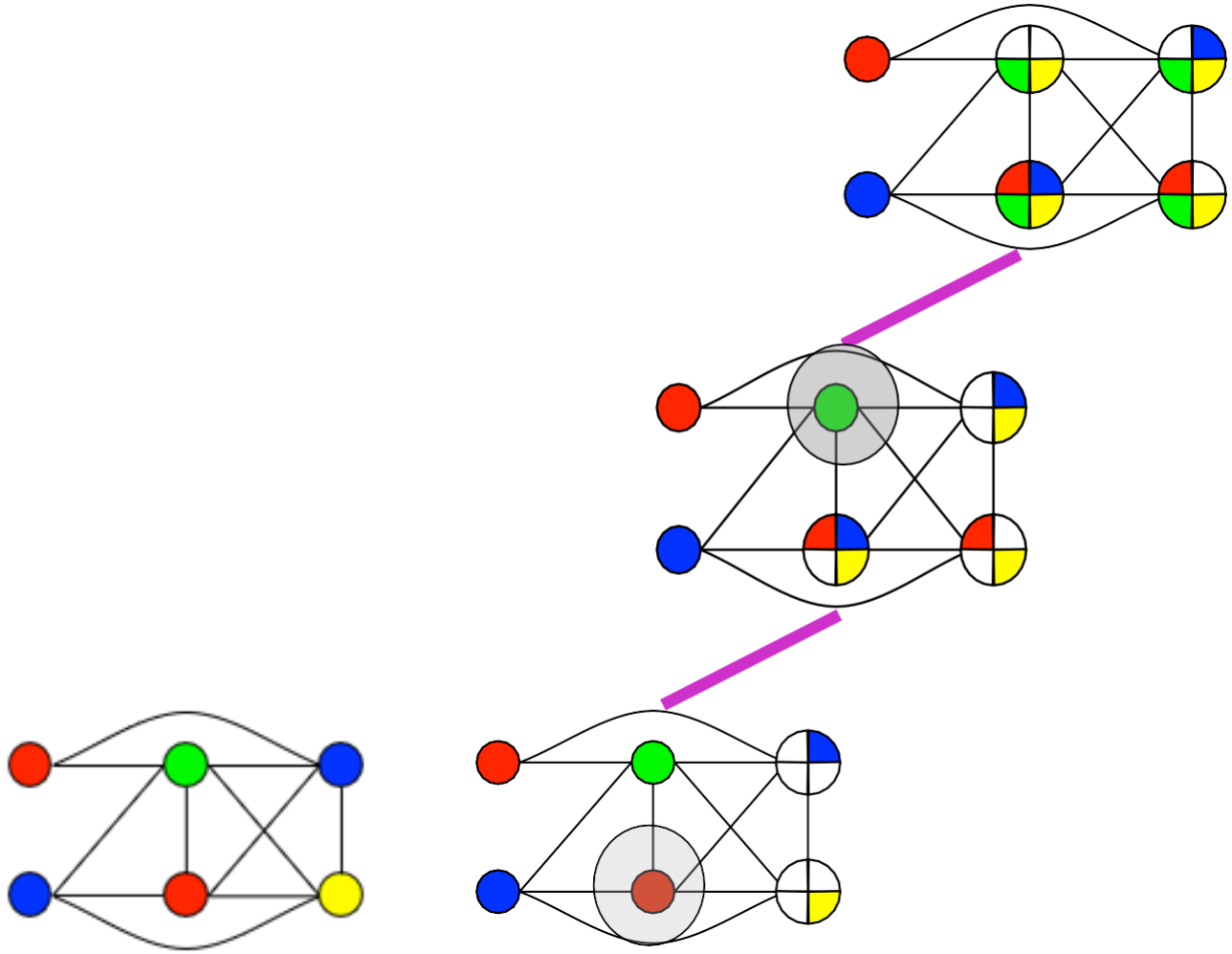
Graph Coloring CP



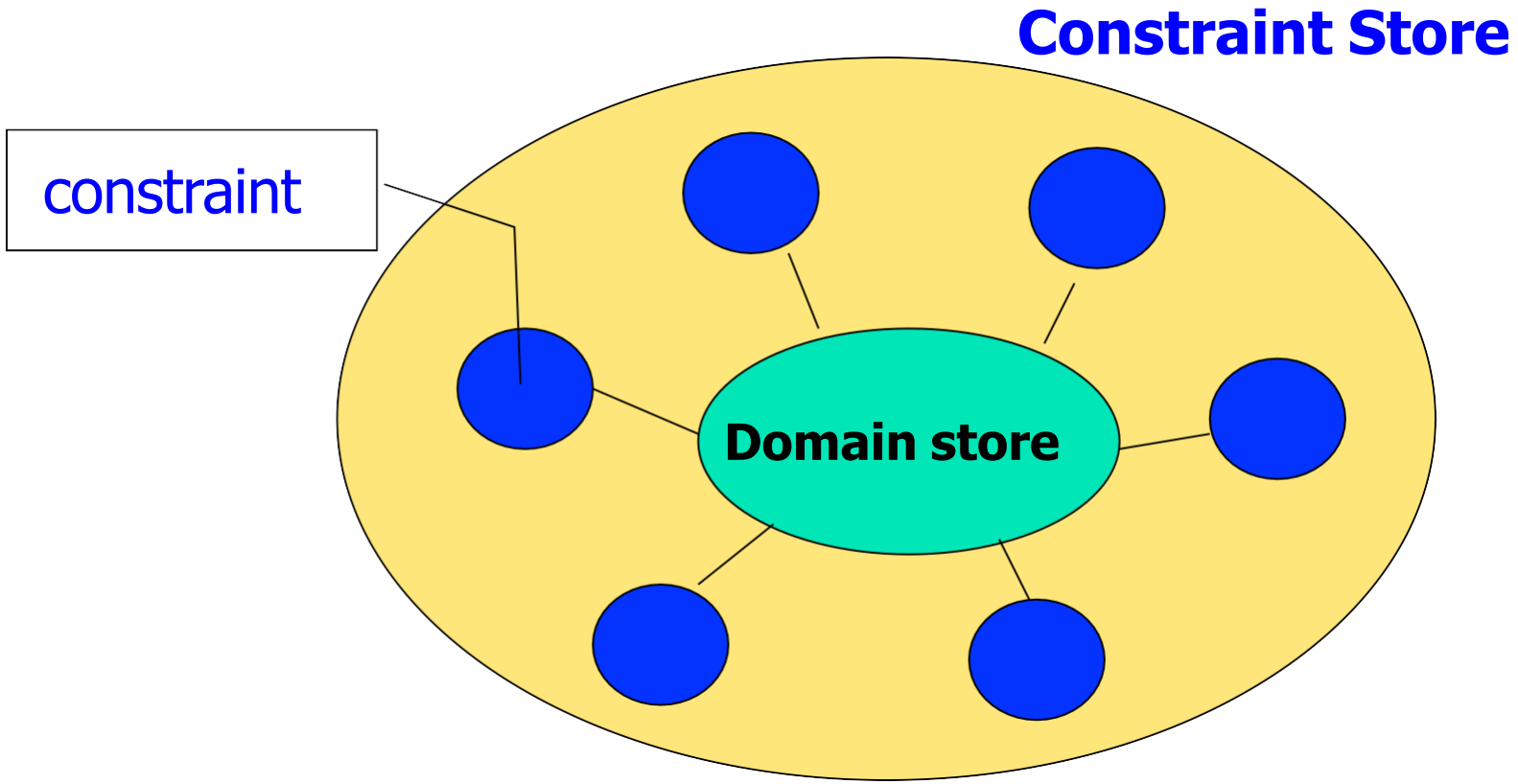
Graph Coloring CP



Graph Coloring CP



Computational Model



- Domain store
  - For each variable: what is the set of possible values?
  - If empty for any variable, then infeasible
  - If singleton for all variables, then solution
- Constraints
  - Capture interesting and well studied substructures
  - Need to
    - **Check Feasibility** : Determine if constraint is feasible wrt the domain store
    - **Prune** : remove "impossible" values from the domains

Constraint Propagation

```
Algorithm AC3(X,D,C){
  Q = {(x,c) | c ∈ C ∧ x ∈X(c)};
  while(Q not empty){
    select and remove (x,c) from Q;
    if ReviseAC3(x,c) then{
      if D(x) = {} then
         return false;
      else
         Q = Q ∪{(x',c') | c'∈C\{c} ∧ x,x' ∈X(c') ∧ x≠ x'}
    }
  }
  return true;
}
```

```
Algorithm ReviseAC3(x,c){
  CHANGE = false;
  for v ∈ D(x) do{
    if there does not exists other values
      of X(c) \ {x} such that c
         is satisfied then{
        remove v from D(x);
        CHANGE = true;
    }
  }
  return CHANGE;
}
```

Content

- A first challenge
- Constraint Satisfaction Problem
- **Constraint Programming**
  - What is CP ?
  - Constraints and propagation
  - **Modeling**
  - Searching
  - Global constraint
- Examples with Or-Tools

Central theme

Solution process
=
Model + Search

- The model
  - what the solutions are and their quality
- The search
  - how to find the solutions

Example : SEND + MORE = MONEY

- Replace each letter by a different digit so that

```
  SEND
+ MORE
------
 MONEY
```

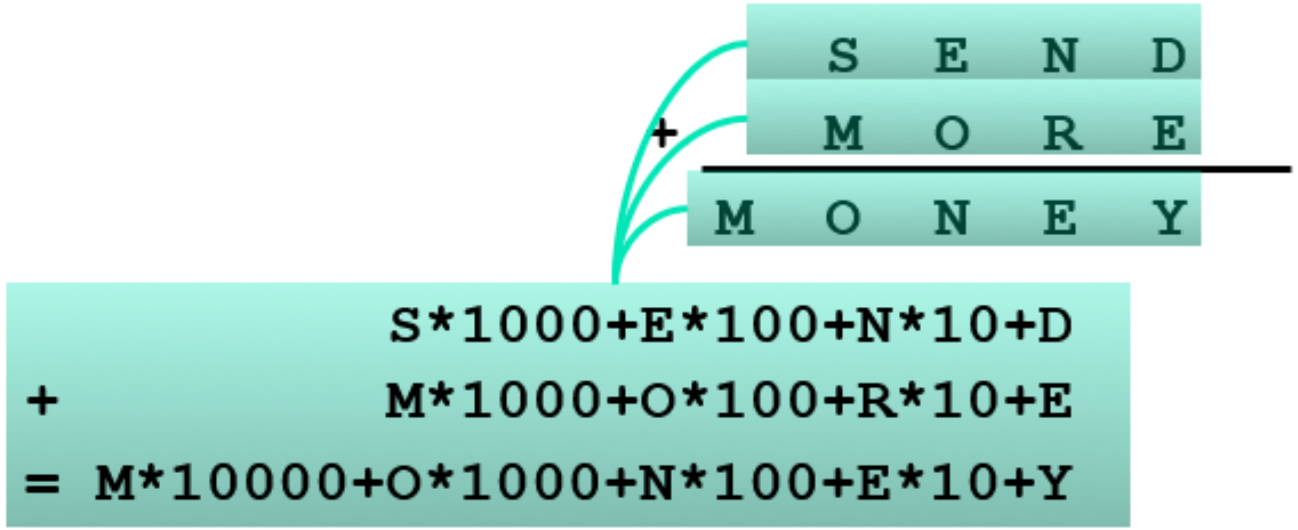
  is a correct sum.
  - Unique solution:

```
  9567
+ 1085
------
 10652
```

- Variables: S, E, N, D, M, O, R, Y
- Domains:
  - [1::9] for S, M,
  - [0::9] for E, N, D, O, R, Y .

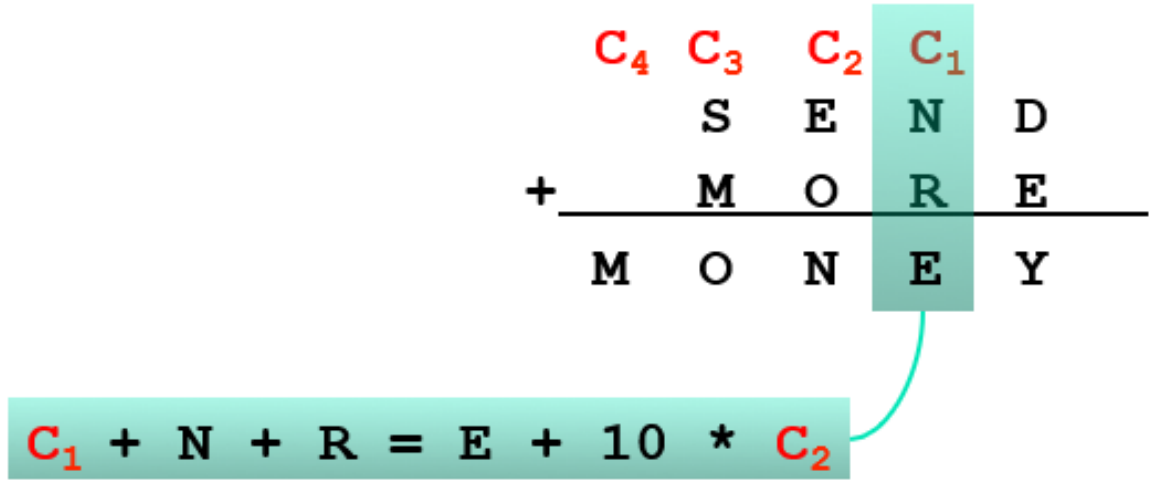
Example : SEND + MORE = MONEY

- 1 equality constraint
  - 1000 S + 100 E + 10 N + D +
    1000 M + 100 O + 10 R + E
    = 10000 M + 1000 O + 100 N + 10 E + Y



Example : SEND + MORE = MONEY

- **Or** 5 equality constraints.
  - Use carry variables C1, …, C4 ∈ [0..1]
  - D + E = 10 C1 + Y
  - C1 + N + R = 10 C2 + E
  - C2 + E + O = 10 C3 + N
  - C3 + S +M = 10 C4 + O
  - C4 = M



Example : SEND + MORE = MONEY

- 28 disequality constraints.
  - $x \neq y$ for $x, y \in \{S;E;N;D;M;O;R;Y\}$
- **Or** a single constraint for disequalities.
  - For variables x1,…, xn with domains D1,…,Dn : all_different(x1, …, xn) := {(d1,…,dn) | $d_i \neq d_j$ for $i \neq j$}
  - Use `all_different(S,E,N,D,M,O,R,Y)`
- **Or** modeling it as an IP problem.
  - For $x, y \in \{S;E;N;D;M;O;R;Y\}$ transform $x \neq y$ to
  - $x - y <= 10 - 11\ z_{xy}$
  - $y - x <= 11\ z_{xy} - 1$
- where $z_{xy} \in [0..1]$; $z_{xy}=1 \Rightarrow x<y$
  - Disadvantage: 28 new variables.

Auxiliary variables

- An auxiliary variables is a decision variable that is not necessary to model the problem
- Motivations:
  - Make it easier to state the problem constraints
  - factorize common expressions and constraints

Redundant constraints

- What are redundant constraints?
  - Semantically redundant: do not exclude any solution
- Motivation
  - Computationally significant: Reduce the search space
- How to find redundant constraints?
  - Express properties of the solutions

Symmetries

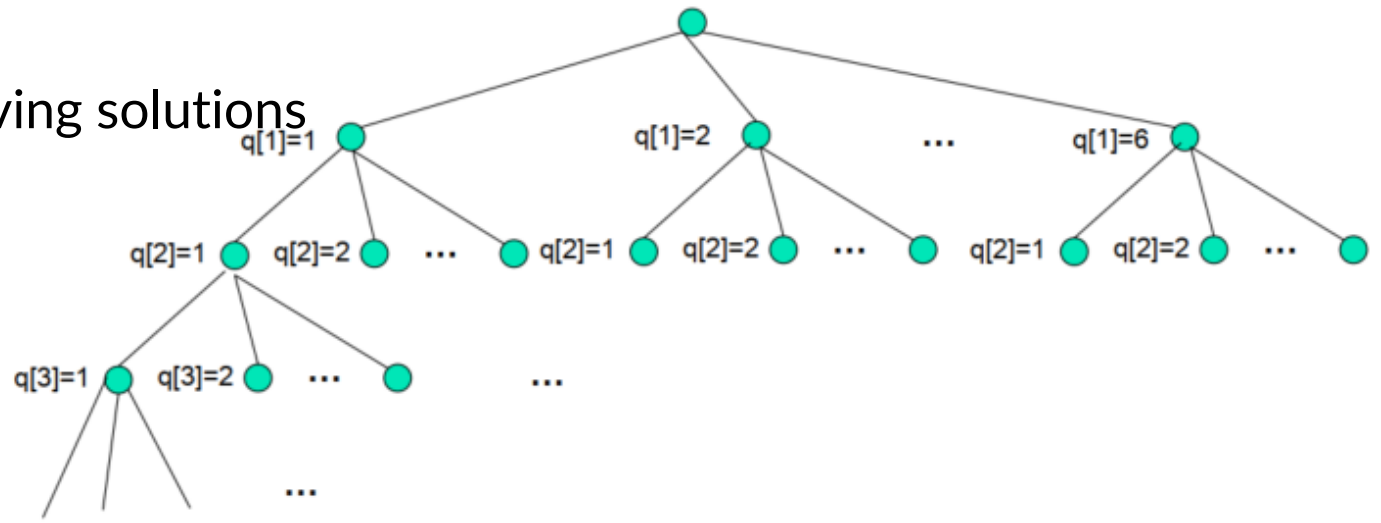
- Many problems naturally exhibit symmetries
  - Exploring symmetrical solution is useless
- Many kinds of symmetries
  - Variable symmetries
  - Value symmetries
- How to break symmetries?
  - Introduce symmetry-breaking constraints

Content

- A first challenge
- Constraint Satisfaction Problem
- **Constraint Programming**
  - What is CP
  - Constraints and propagation
  - Modeling
  - **Searching**
  - Global constraint
  - Computation domain
- Examples with Or-Tools

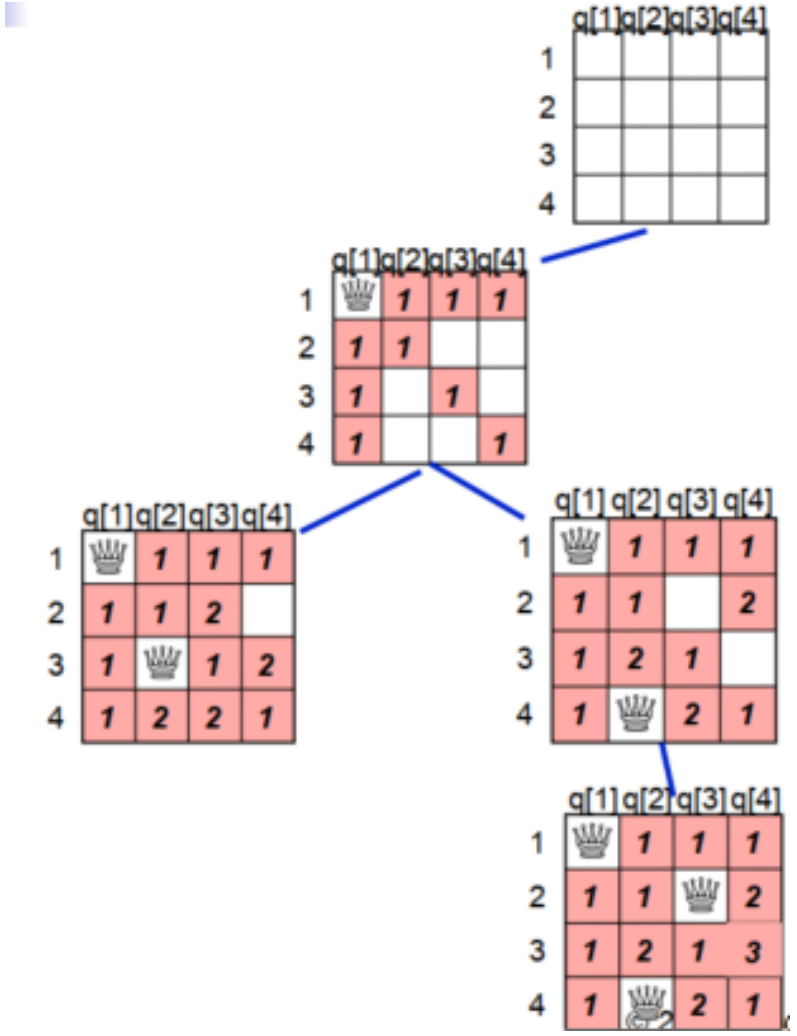
Necessity of search

- Propagation techniques are not aimed at solving a problem
- Necessity to combine the propagation with a search mechanism
- Search tree of a CSP
  - The root is the given the CSP
  - The nodes are CSPs
  - If $P_1 \ldots, P_m$ are direct descendant of P_0, then the union of $P_1 \ldots, P_m$ is equivalent to P_0
- Propagation
  - Propagation is performed at each node
  - Reduction of the domains, without moving solutions
  - The resulting CSP is equivalent
- Leaves of the tree
  - Solution node
  - Failed node



Search with trailing

- Trailing: method allowing to undo operation performed on data structures
- Reconstruct a previous state of computation
- No copy of the domains: operation on domains are trailed
- Undo operation performed at the end of the algorithm to restore the domains
- Abstract global object that trails variables assignments and value removals in domains
- Trailed operations are labeled with an integer: the trail level
- Trail level will be the depth of the current node.



Branching

- Once constraint propagation is done, apply the search method

```
Choose a variable x with non-singleton domain
       (d1,  d2, … di)
For each d in (d1, d2, … di)
  try to solve the problem with the
  additional constraint x=d
```

Branching strategy

- var+val labeling
  - choose a variable x
  - one child per value in D(x)
  - depth of search tree is n
- var+val binary labeling
  - choose a variable x, and a value a in D(x)
  - one child with x=a, one child with x"a
  - depth of the search tree is larger than n
- var+val domain splitting
  - choose a variable x, and a value a in D(x)
  - one child with x≤ a, one child with x>a
  - depth of the search tree is larger than n

Branching heuristics

- How to choose the variable ?
  - Variable ordering heuristics
- How to order the values ?
  - Value ordering heuristics
- Static heuristics
  - Computed once at the beginning of the search
- Dynamic heuristics
  - Computed on the CSP of the current node

Minimal tree principle

- Make a choice that is likely to reduce the size of the search tree (independently of the propagation)
- The first-fail principle
  - Branch on a variable that once instantiated, yields a CSP whose failure is most likely to be detected early
  - First consider the variable belonging to the most difficult part of the CSP
  - Very useful when the CSP has no solution
  - Detect the failure without considering all variables
- The best-first principle
  - Choose a variable that is most likely to lead to a solution
  - Choose a value that is not in a solution is useless
  - To find a solution, all the variables do not have to be considered

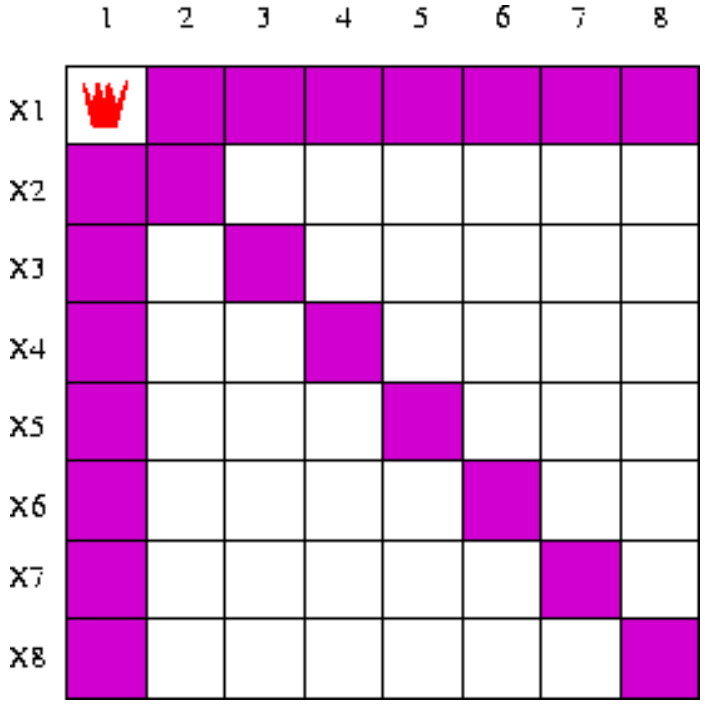
Variable ordering heuristics

- Lexico
  - Lexicographic order
  - Static
  - Often used as a default
- Dom
  - Select variable with the smallest domain
  - Dynamic
  - Minimum tree principle

The Queen Problem

Assuming domain consistency

Propagation of
queen[1] <> queen[2]
queen[1] <> queen[3]
…
queen[1] <> queen[8]

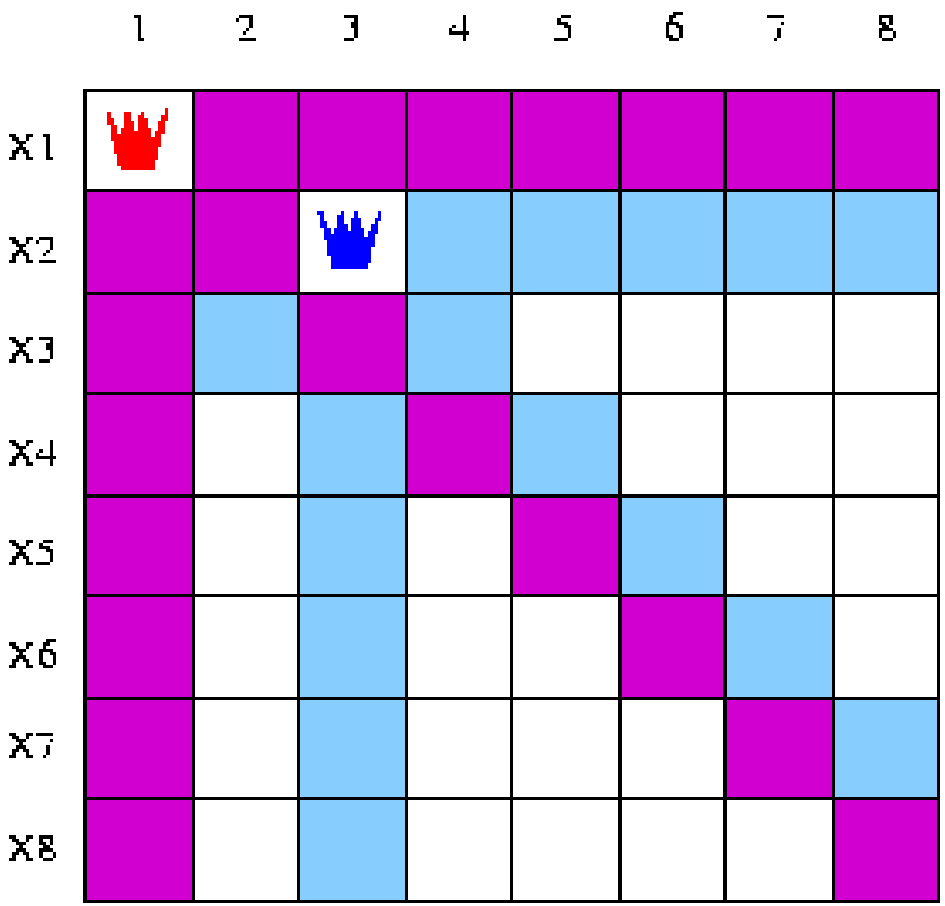


Consequence of
queen[1]=1

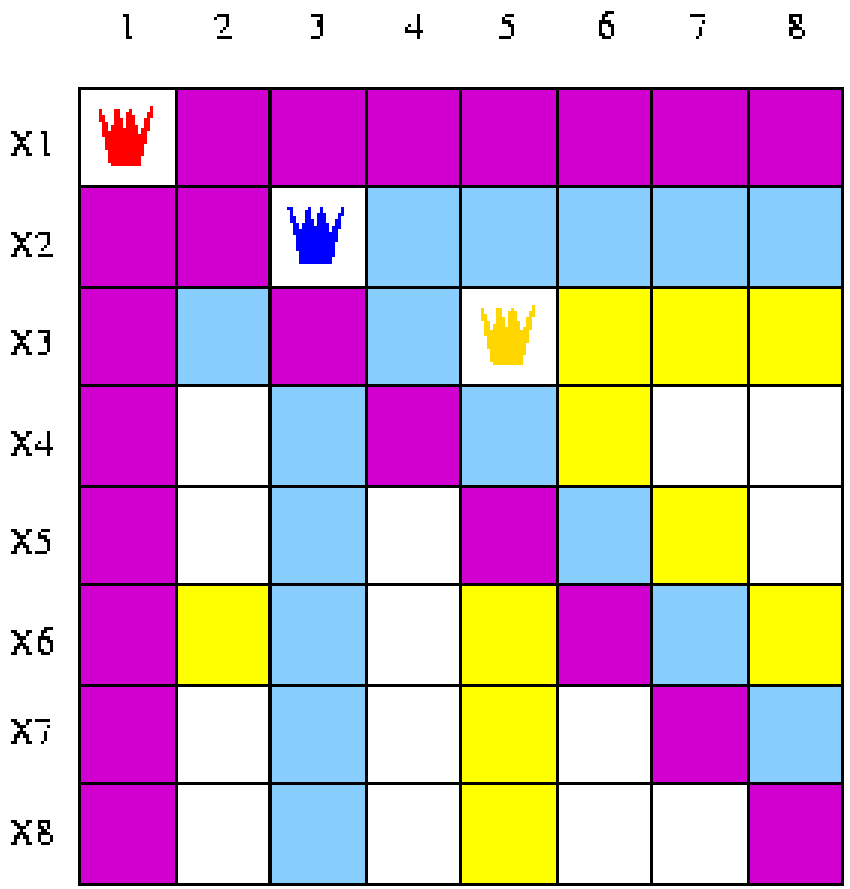
Propagation of
queen[1]+1<> queen[2]+2
queen[1]+1<> queen[3]+3
…
queen[1]+1 <> queen[8]+8

No more pruning
Place another queen
Questions
Which one ?
On which tile ?

The Queen Problem



The Queen Problem



Content

- A first challenge
- Constraint Satisfaction Problem
- **Constraint Programming**
  - What is CP ?
  - Constraints and propagation
  - Modeling
  - Searching
  - **Global constraint**
- Examples with Or-Tools

Global Constraints

- Recognize some combinatorial substructures arising in many practical applications
  - alldifferent is a fundamental building block
  - many others (as we will see)
- Make modeling easier and more natural
  - Declarative
  - Compositionality

The alldifferent Constraint

- Most well-known global constraint.
  - `alldifferent(x,y,z)`
- states that x, y, and z take on different values.
  - x=2, y=1, z=3 would be ok,
  - but not x=1, y=3, z=1
- Why does it prune more than x≠y, y≠z, x≠z ?
- Very useful in many resource allocation, time tabling, sport scheduling problems

Content

- A first challenge
- Constraint Satisfaction Problem
- Constraint Programming
  - What is CP ?
  - Constraints and propagation
  - Modeling
  - Searching
  - Global constraint
- **Examples with Or-Tools**

Example 1

- **Variables**
  - X = {X0, X1, X2, X3, X4}
- **Domains of variables**
  - X0, X1, X2, X3, X4 ∈ {1,2,3,4,5}
- **Constraints**
  - C1: X2 + 3 ≠ X1
  - C2: X3 ≤ X4
  - C3: X2 + X3 = X0 + 1
  - C4: X4 ≤ 3
  - C5: X1 + X4 = 7
  - C6: X2 = 1 => X4 ≠ 2

Solution

```python
from ortools.sat.python import cp_model

model = cp_model.CpModel()

x0 = model.NewIntVar(1, 5, 'x0')
x1 = model.NewIntVar(1, 5, 'x1')
x2 = model.NewIntVar(1, 5, 'x2')
x3 = model.NewIntVar(1, 5, 'x3')
x4 = model.NewIntVar(1, 5, 'x4')

model.Add(x2 + 3 != x1)
model.Add(x3 <= x4)
model.Add(x2 + x3 == x0 + 1)
model.Add(x4 <= 3)
model.Add(x1 + x4 == 7)

b = model.NewBoolVar('b')
model.Add(x2 == 1).OnlyEnforceIf(b)
model.Add(x2 != 1).OnlyEnforceIf(b.Not())
model.Add(x4 != 2).OnlyEnforceIf(b)

solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.OPTIMAL:
    print('Maximum of objective function: %i' % solver.ObjectiveValue())
    print()
    print('x0 value: ', solver.Value(x0))
    print('x1 value: ', solver.Value(x1))
    print('x2 value: ', solver.Value(x2))
    print('x3 value: ', solver.Value(x3))
    print('x4 value: ', solver.Value(x4))
```

Example 2: CP + IS + FUN = TRUE

```
def CPIsFunSat():
    """Solve the CP+IS+FUN==TRUE cryptarithm."""
    # Constraint programming engine
    model = cp_model.CpModel()

    base = 10

    c = model.NewIntVar(1, base - 1, 'C')
    p = model.NewIntVar(0, base - 1, 'P')
    i = model.NewIntVar(1, base - 1, 'I')
    s = model.NewIntVar(0, base - 1, 'S')
    f = model.NewIntVar(1, base - 1, 'F')
    u = model.NewIntVar(0, base - 1, 'U')
    n = model.NewIntVar(0, base - 1, 'N')
    t = model.NewIntVar(1, base - 1, 'T')
    r = model.NewIntVar(0, base - 1, 'R')
    e = model.NewIntVar(0, base - 1, 'E')

    # We need to group variables in a list to use the constraint AllDifferent.
    letters = [c, p, i, s, f, u, n, t, r, e]
```

Example 2: CP + IS + FUN = TRUE

```
# Verify that we have enough digits.
assert base >= len(letters)

# Define constraints.
model.AddAllDifferent(letters)

# CP + IS + FUN = TRUE
model.Add(c * base + p + i * base + s + f * base * base + u * base +
          n == t * base * base * base + r * base * base + u * base + e)

### Solve model.
solver = cp_model.CpSolver()
solution_printer = VarArraySolutionPrinter(letters)

# Enumerate all solutions.
solver.parameters.enumerate_all_solutions = True
# Solve.
status = solver.Solve(model, solution_printer)

print()
print('Statistics')
print('  - status          : %s' % solver.StatusName(status))
print('  - conflicts       : %i' % solver.NumConflicts())
print('  - branches        : %i' % solver.NumBranches())
print('  - wall time       : %f s' % solver.WallTime())
print('  - solutions found : %i' % solution_printer.solution_count())
```

Example 2: CP + IS + FUN = TRUE

```
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self):
        self.__solution_count += 1
        for v in self.__variables:
            print('%s=%i' % (v, self.Value(v)), end=' ')
        print()

    def solution_count(self):
        return self.__solution_count
```

Example 3: N-Queen problem

```
def main(board_size):
  model = cp_model.CpModel()
  # Creates the variables.
  # The array index is the column, and the value is the row.
  queens = [model.NewIntVar(0, board_size - 1, 'x%i' % i)
            for i in range(board_size)]
  # Creates the constraints.
  # The following sets the constraint that all queens are in different rows.
  model.AddAllDifferent(queens)

  # Note: all queens must be in different columns because the indices
  #of queens are all different.
```

Example 3: N-Queen problem

```
# The following sets the constraint that no two queens can be on
# the same diagonal.
for i in range(board_size):
  # Note: is not used in the inner loop.
  diag1 = []
  diag2 = []
  for j in range(board_size):
    # Create variable array for queens(j) + j.
    q1 = model.NewIntVar(0, 2 * board_size, 'diag1_%i' % i)
    diag1.append(q1)
    model.Add(q1 == queens[j] + j)
    # Create variable array for queens(j) - j.
    q2 = model.NewIntVar(-board_size, board_size, 'diag2_%i' % i)
    diag2.append(q2)
    model.Add(q2 == queens[j] - j)
  model.AddAllDifferent(diag1)
  model.AddAllDifferent(diag2)
```

Example 3: N-Queen problem

```
  ### Solve model.
  solver = cp_model.CpSolver()
  solution_printer = SolutionPrinter(queens)
  status = solver.SearchForAllSolutions(model, solution_printer)
  print()
  print('Solutions found : %i' % solution_printer.SolutionCount())


class SolutionPrinter(cp_model.CpSolverSolutionCallback):
  """Print intermediate solutions."""

  def __init__(self, variables):
    cp_model.CpSolverSolutionCallback.__init__(self)
    self.__variables = variables
    self.__solution_count = 0

  def OnSolutionCallback(self):
    self.__solution_count += 1
    for v in self.__variables:
      print('%s = %i' % (v, self.Value(v)), end = ' ')
    print()

  def SolutionCount(self):
    return self.__solution_count
```

Example 4: Class Allocation Problem

- n classes labeled by $1, 2, \ldots, n$ need to be allocated in p semesters $1, 2, \ldots p$. Each class i has a credit value of $c(i)$, and its prerequisite conditions are defined by a set Q of pairs (i, j), where subject i must be taken before j. Given the constant $\alpha, \beta, \delta, \gamma$, it is necessary to determine an allocation plan that satisfies the following:
  - The total number of classes assigned to each semester must be greater than or equal to α and less than or equal to β.
  - The total number of credits of the classes assigned to each semester must be greater than or equal to δ and less than or equal to γ
  - For each pair $(i, j) \in Q$, class i must be scheduled in a semester prior to the semester in which class j is scheduled.
- Objective: The maximum number of credits in any one semester must be minimized.

Example 4: Class Allocation Problem

- Example

Class	1	2	3	4	5	6	7	8	9	10	11	12
Number of credits	2	1	2	1	3	2	1	3	2	3	1	3

3 ≤ Number of classes in each semester ≤ 3
5 ≤ Number of credits in each semester ≤ 7

Set Q

2	1
6	9
5	6
5	8
4	11
6	12
2	7
3	10
5	7
8	11
4	12

Example 4: Class Allocation Problem

- Example

Class	1	2	3	4	5	6	7	8	9	10	11	12
Number of credits	2	1	2	1	3	2	1	3	2	3	1	3

3 ≤ Number of classes in each semester ≤ 3

5 ≤ Number of credits in each semester ≤ 7

Allocation solution

Semester	1	2	3	4
List of classes	2, 5, 3	1, 6,10	4,7,8	9,11,12

Set Q

2	1
6	9
5	6
5	8
4	11
6	12
2	7
3	10
5	7
8	11
4	12

Example 4: Class Allocation Problem

- Input:
  - Set of classes: $C = \{1, \dots, n\}$
  - Set of semesters: $S = \{1, \dots, p\}$
  - Set of precedent classes: $Q = \{(i_1, j_1), \dots, (i_k, j_k) | i_1, \dots, i_k, j_1, \dots, j_k \in C, i_t \neq j_t \ \forall t \in \{1, \dots, k\}\}$
  - Constant $\alpha, \beta, \delta, \gamma$
- Variables:
  - Binary variable x_{ij} $(i \in C, j \in S)$ is equal to 1 if the class i is assigned to the semester j, otherwise the value of this variable is equal to 0;
  - Integral variable $maxcredit$ represents the maximum number of credits for a semester
- Constraints:
  - Every class is allocated to some semester $\sum_{j \in S} x_{ij} = 1, \forall i \in C$
  - Number of classes in a semester must be in a range of $[\alpha, \beta]$, it means that $\alpha \leq \sum_{i \in C} x_{ij} \leq \beta, \forall j \in S$
  - Number of credits in a semester must be in a range of $[\delta, \gamma]$, it means that $\delta \leq \sum_{i \in C} c(i) x_{ij} \leq \gamma$
  - For each pair $(i_1, i_2) \in Q$, class i_1 must be scheduled in a semester prior to the semester in which class i_2 is scheduled $\sum_{j \in S} j x_{i_1 j} < \sum_{j \in S} j \, x_{i_2 j}$, $\forall (i_1, i_2) \in Q$
  - Relation between variable $maxcredit$ and workload in a semester $\sum_{i \in C} c(i) x_{ij} \leq maxcredit$, $\forall j \in$S
- Objective: $Minimize \ maxcredit$

Example 4: Bài toán Phân bổ môn học

```
N = 12
P = 4
credits = [2, 1, 2, 1, 3, 2, 1, 3, 2, 3, 1, 3]

orderCourseLst = [(0, 1), (1, 2)]

alpha = 3
beta = 3
lamb = 5
gamma = 7

model = cp_model.CpModel()

# Khai báo biến x[i,j] = 1 nếu môn học j được phân vào kỳ p
x = []
for i in range(P):
    t = []
    for j in range(N):
        t.append(model.NewIntVar(0, 1, 'x['+ str(i) + "," + str(j) + "]"))
    x.append(t)
```

Example 4: Bài toán Phân bổ môn học

```
#Ràng buộc: Mỗi môn học chỉ được phân vào duy nhất 1 kỳ
for j in range(N):
    model.Add(sum(x[i][j] for i in range(P)) == 1)

#Ràng buộc: Số lượng môn học trong một kỳ phải nằm trong [alpha, beta]
for i in range(P):
    model.Add(sum(x[i][j] for j in range(N)) >= alpha)
    model.Add(sum(x[i][j] for j in range(N)) <= beta)

#Ràng buộc: Số tín chỉ trong một kỳ phải nằm trong [lambda, gamma]
for i in range(P):
    model.Add(sum(x[i][j]*credits[j] for j in range(N)) >= lamb)
    model.Add(sum(x[i][j]*credits[j] for j in range(N)) <= gamma)
```

Example 4: Bài toán Phân bổ môn học

```
#Ràng buộc: Thứ tự các môn học
for item in orderCourseLst:
    i = item[0]
    j = item[1]

    for q in range(P):
        for p in range(q+1):

            b = model.NewBoolVar('b')
            model.Add(x[q][i] == 1).OnlyEnforceIf(b)
            model.Add(x[q][i] != 1).OnlyEnforceIf(b.Not())
            model.Add(x[p][j] == 0).OnlyEnforceIf(b)
```

Example 4: Bài toán Phân bổ môn học

```
### Solve model.
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f'Total cost = {solver.ObjectiveValue()}')
    print()
    for i in range(P):
        for j in range(N):
            if solver.BooleanValue(x[i][j]):
                print(
                    f'Môn học {j} được phân cho kỳ {i}')
else:
    print('No solution found.')
```

Minimal Length Edge Disjoint Paths

- Given a network G = (V,E). Each edge (i,j) has length c(i,j). Find two edge-disjoint paths from 1 to n (two paths that do not have common edges) such that the sum of lengths of the two paths is minimal.

- **Input**
- Line 1: contains 2 positive integers n and m (1 <= n,m <= 30)
- Line i+1 (i = 1, 2, . . ., m): contains 3 positive integers u, v, c in which c is the length of the edge(u,v)

- **Output**
- Write the sum of lengths of the two edge-disjoint paths found, or write NOT_FEASIBLE if no solution found.

Minimal Length Edge Disjoint Paths

```python
#PYTHON
import sys
from ortools.sat.python import cp_model

def Input():
 [n,m] = [int(x) for x in sys.stdin.readline().split()]
 E = []

 InA = [[] for i in range(n+1)]
 OutA = [[] for j in range(n+1)]
 #for i in range(n+1):
 # InA[i] = []
 # OutA[i] = []

 for i in range(m):
  [u,v,w] = [int(x) for x in sys.stdin.readline().split()]
  OutA[u].append([v,w])
  InA[v].append([u,w])
  E.append([u,v,w])
  E.append([v,u,w])

 return n,m,InA,OutA,E
```

```python
n,m,InA,OutA,E = Input()

model = cp_model.CpModel()

#define decision variables
x = {}
y = {}
#Lx = {}
#Ly = {}

for [i,j,w] in E:
 x[i,j] = model.NewIntVar(0,1,'x(' + str(i) + ',' + str(j) + ')')
 y[i,j] = model.NewIntVar(0,1,'y(' + str(i) + ',' + str(j) + ')')

MAX = 0
for [i,j,w] in E:
 MAX = MAX + w
MAX = MAX//2

obj = model.NewIntVar(0,MAX,'obj')

s = 1
t = n
```

Minimal Length Edge Disjoint Paths

```
# constraints
for i in range(1,n+1):
 if i != s and i != t:
  model.Add(sum(x[j,i] for [j ,w] in InA[i]) == sum(x[i,j] for [j ,w] in OutA[i]))
  model.Add(sum(y[j,i] for [j ,w] in InA[i]) == sum(y[i,j] for [j ,w] in OutA[i]))

model.Add(sum(x[s,j] for [j,w] in OutA[s]) == 1)
model.Add(sum(x[j,t] for [j,w] in InA[t]) == 1)
model.Add(sum(y[s,j] for [j,w] in OutA[s]) == 1)
model.Add(sum(y[j,t] for [j,w] in InA[t]) == 1)
```

```
# disjoint constraint x(i,j) + y(i,j) <= 1
for [i,j,w] in E:
 model.Add(x[i,j] + y[i,j] <= 1)

# objective function
#model.Minimize(Lx[t] + Ly[t])
obj = 0
for [i,j,w] in E:
 obj += (x[i,j] + y[i,j])*w
model.Minimize(obj)


solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 5.0
status = solver.Solve(model)

if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
 print(solver.Value(obj))
```

DELAY_CONSTRAINT_BROADCAST_TREE

Given a network V = {1,..., N} is the set of nodes, E in VxV is the set of links between nodes.

A node s in V is the source node which will transmit a package to others nodes.

A node receiving the package can continue transmit this package to adjacent nodes.

t(i,j) and c(i,j) are transmission time and transmission cost when transmitting the package from node i to node j.

Compute the set of links used for broadcasting the package from the source node to all other nodes such that

- Total transmission time from s to any node cannot exceed a given value L
- Total transmission cost is minimal

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !